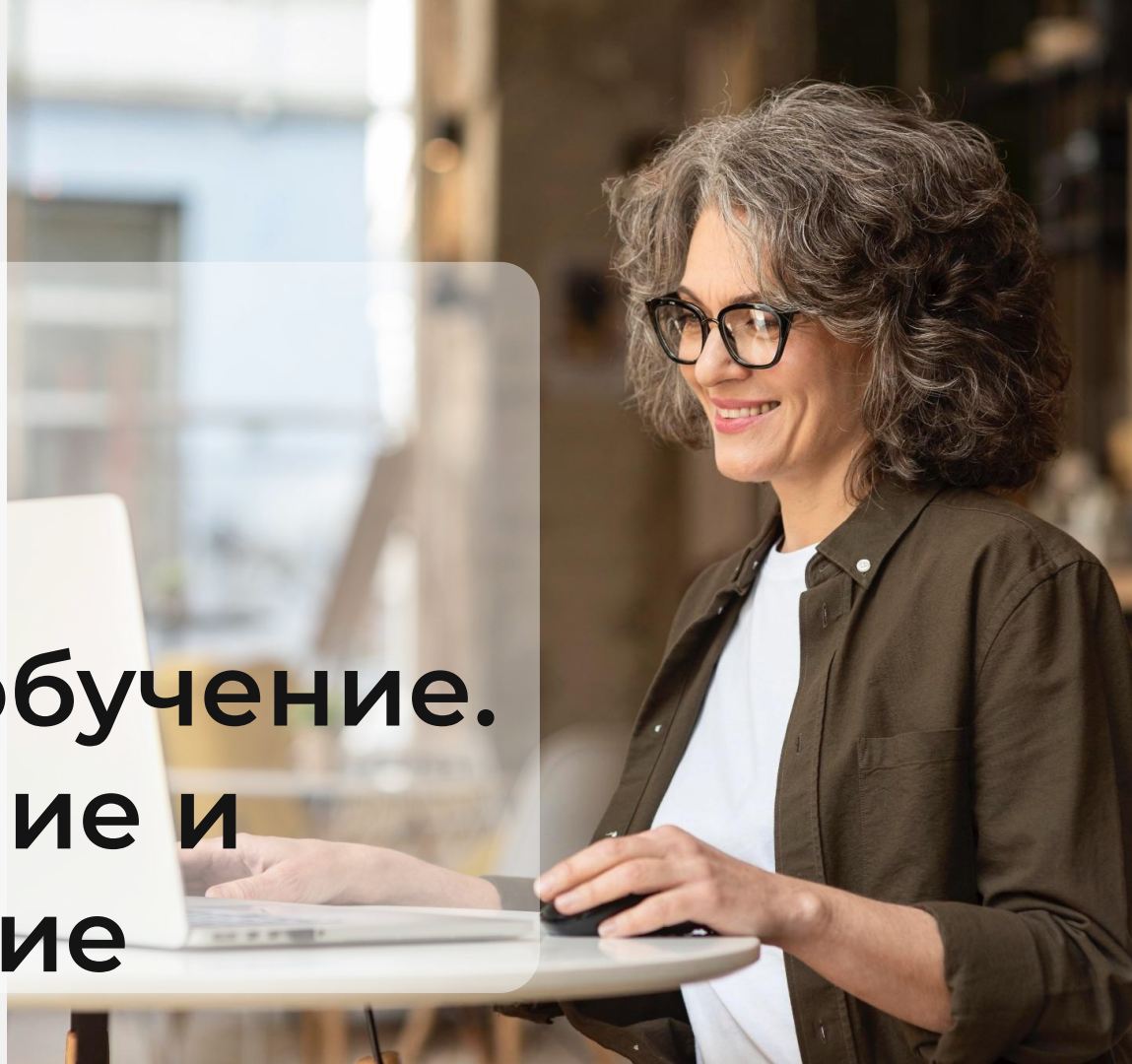


Python
for DA

Введение в машинное обучение. Переобучение и недообучение



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Введение в оценку моделей
-  Accuracy: Определение и расчет
-  Accuracy: реализация на Python
-  Precision и Recall: Определения и расчеты
-  Компромисс между показателями Precision и Recall
-  Кривая Precision-Recall
-  F1-Score: Определение и расчет

План занятия

- Понимание оверфиттинга и андерфиттинга
- Диагностика переобучения и недообучения
- Диагностика избыточной и недостаточной подгонки
- Регуляризация: регрессии Лассо и Риджа
- Реализация регуляризации (L1, L2) на основе данных



ОСНОВНОЙ БЛОК





Понимание оверфиттинга и андерфиттинга



Цель машинного обучения

Это разработка моделей, которые хорошо обобщают новые, неизвестные данные. Достичь этого баланса бывает непросто, и модели могут страдать от двух распространенных проблем: переобучения и недообучения.

Причины переобучения

-  Сложные модели
-  Недостаточное количество обучающих данных
-  Шум в данных
-  Отсутствие регуляризации
-  Слишком много признаков

Причины недообучения



Простые модели



Недостаточное время обучения



Неадекватные признаки



Высокая регуляризация



Неправильный выбор модели

Стратегии предотвращения переобучения

Кросс-валидация

Регуляризация

Обрезка

Ранняя остановка

Ансамблевые методы

Стратегии предотвращения недообучения

Повышайте сложность
моделей

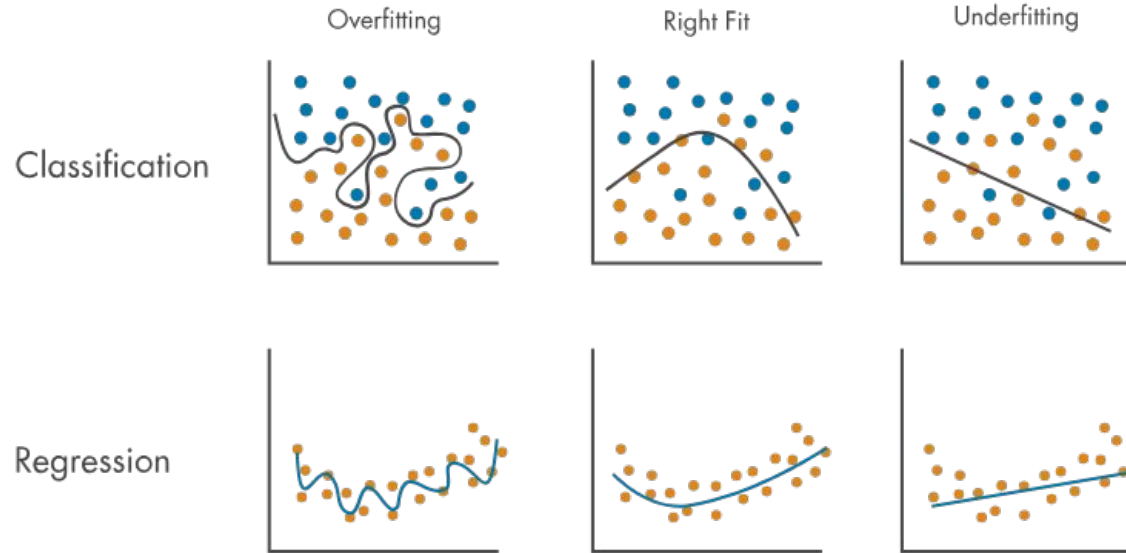
Инженерия признаков

Снижение регуляризации

Обучать дольше

Используйте подходящие
модели

Overfitting и underfitting





ВОПРОСЫ





Диагностика переобучения и недообучения



Кривые обучения

Это графические представления, иллюстрирующие, как меняется производительность модели с ростом опыта, обычно измеряемого количеством обработанных ею обучающих данных.

Избыточная и недостаточная подгонка с помощью кривых обучения



Переобучение

- Точность обучения высока и остается стабильной или даже растет.
- Точность проверки значительно ниже точности обучения и может даже снижаться со временем.
- Модель запоминает учебные данные, а не изучает общие закономерности.

Недообучение

- Точность обучения и проверки низкая и остается относительно постоянной.
- Модель не способна уловить существенные особенности данных, что приводит к низкой производительности на обоих наборах.



Функция `learning_curve` в Scikit-learn

Это инструмент для оценки производительности модели машинного обучения по мере увеличения размера обучающего набора данных.

Ключевые параметры: estimator



Тип

Объект

Описание

Модель машинного обучения или оценщик, который будет использоваться. Этот объект должен реализовывать методы fit и predict.

Ключевые параметры: X



Тип

array-like, форма (n_samples, n_features)

Описание

Входные данные, которые будут использоваться для обучения модели.

Ключевые параметры: y



Тип

array-like, форма (n_образцов) или (n_образцов, n_функций)

Описание

Целевые значения (метки классов в классификации, вещественные числа в регрессии).

Ключевые параметры: train_sizes



Тип

array-like, форма (n_ticks,)

Описание

Относительное или абсолютное количество обучающих примеров, которые будут использованы для построения кривой обучения. Если тип - float, оно рассматривается как дробь от максимального размера обучающего набора.

По умолчанию: `np.linspace(0.1, 1.0, 5)`

Ключевые параметры: cv



Тип

int, генератор кросс-валидации или итерируемая переменная

Описание

Определяет стратегию разбиения кросс-валидации. Возможными значениями для cv являются:

- None, чтобы использовать 5-кратную кросс-валидацию по умолчанию.
- Целое число, чтобы указать количество наборов.

Ключевые параметры: scoring



Тип

string, callable или None

Описание

Строка или вызываемый объект/функция scorer с сигнатурой scorer(estimator, X, y). Если None, то используется оценщик по умолчанию (если он есть).

Пример



Диагностика переобучения и недообучения с помощью кривых обучения в Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve, train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.datasets import fetch_california_housing
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузите набор данных
data = fetch_california_housing()
X, y = data.data, data.target

# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Шаг 3: Определите функцию для построения кривых обучения

```
def plot_learning_curves(estimator, X, y, train_sizes=np.linspace(0.1, 1.0, 10), cv=5):
    train_sizes, train_scores, validation_scores = learning_curve(
        estimator, X, y, train_sizes=train_sizes, cv=cv, scoring='neg_mean_squared_error'
    )

    train_scores_mean = -train_scores.mean(axis=1)
    validation_scores_mean = -validation_scores.mean(axis=1)

    plt.figure(figsize=(12, 6))
    plt.plot(train_sizes, train_scores_mean, 'o-', color='r', label='Training error')
    plt.plot(train_sizes, validation_scores_mean, 'o-', color='g', label='Validation error')
    plt.xlabel('Размер обучающих данных')
    plt.ylabel('Ошибка')
    plt.title('Кривые обучения')
    plt.legend(loc='best')
    plt.grid()
    plt.show()
```

Шаг 4: Обучение модели и построение кривых обучения

```
# Инициализируем модель дерева решений
model = DecisionTreeRegressor(max_depth=5, random_state=42)

# Построить кривые обучения
plot_learning_curves(model, X_train, y_train)
```

Интерпретация кривых обучения

- Если кривые обучения показывают, что ошибка обучения постоянно уменьшается и приближается к нулю, а ошибка проверки остается значительно выше и может даже увеличиться, это свидетельствует о переобучении.
- Если кривые обучения показывают, что ошибки обучения и проверки высоки и остаются относительно постоянными, это свидетельствует о недообучении.
- Хорошо подогнанная модель имеет кривые обучения, на которых ошибки обучения и проверки уменьшаются и сходятся к одинаковому значению.



ВОПРОСЫ





Диагностика избыточной и недостаточной подгонки



Кросс-валидация

Это техника, используемая для оценки производительности модели машинного обучения и диагностики таких проблем, как переобучение и недообучение. Разбивая данные на несколько подмножеств и обучая модель на различных комбинациях этих подмножеств, кросс-валидация позволяет получить более полную оценку эффективности модели.

Пример



Диагностика переобучения и недообучения с помощью перекрестной валидации в Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузить набор данных
data = load_breast_cancer()
X, y = data.data, data.target

# Преобразуем в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

Шаг 3: Выполните кросс-валидацию

```
# Инициализируем модель дерева решений
model = DecisionTreeClassifier(random_state=42)

# Выполняем 5-кратную перекрестную валидацию
cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')

# Выведите оценки кросс-валидации
print(f'Cross-Validation Scores: {cv_scores}')
print(f'Средняя оценка кросс-валидации: {cv_scores.mean()}')
```

Шаг 4: Анализ результатов

```
# Разделите данные на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Обучите модель на обучающих данных
model.fit(X_train, y_train)

# Делаем предсказания на обучающем и тестовом наборах
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Рассчитайте точность на обучающем и тестовом наборах
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f'Training Accuracy: {train_accuracy}')
```



ВОПРОСЫ





Регуляризация: регрессии Лассо и Риджа



Регуляризация

Это техника, используемая в машинном обучении для предотвращения чрезмерной подгонки путем добавления штрафа к сложности модели. Этот штраф не позволяет модели подстраиваться под шум в обучающих данных, тем самым улучшая ее обобщение на новые, неизвестные данные.

Методы регуляризации

Lasso (L1) регрессия

Ridge (L2) регрессия



Цель регуляризации

Снизить сложность модели за счет штрафов за большие коэффициенты.

Регрессия LASSO



Определение

Регрессия LASSO или регуляризация L1, добавляет штраф, равный абсолютному значению коэффициентов, к функции стоимости.

Формула

$$\text{Функция потерь} = \frac{1}{2} \sum (y - \hat{y})^2 + \lambda \sum |w|,$$

где:

- y - фактическое значение,
- $\hat{y}$ - предсказанное значение,
- w обозначает коэффициенты признаков,
- λ - параметр регуляризации, который управляет силой штрафа.

Ридж-Регрессия



Определение

Ридж-регрессия или регуляризация L2, добавляет к функции потерь штраф, равный квадрату коэффициентов.

Формула

$$\text{Функция потерь} = \frac{1}{2} \sum (y - \hat{y})^2 + \lambda \sum w^2$$

где:

- y - фактическое значение,
- $\hat{y}$ - предсказанное значение,
- w обозначает коэффициенты признаков,
- λ - параметр регуляризации, который управляет силой штрафа.

Влияние на модель



Уменьшение перебора



Включение всех признаков



Улучшенная стабильность

Сравнение гребневой регрессии и регрессии LASSO



Штрафной компонент



Выбор признаков



Примеры использования



ВОПРОСЫ





Реализация регуляризации (L1, L2) на основе данных

Пример



Применим регрессию LASSO и ридж-регрессию, используя набор данных California Housing.
Этот набор данных содержит информацию о различных характеристиках домов
в Калифорнии и соответствующих им ценах

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso, Ridge
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузка набора данных
data = fetch_california_housing()
X, y = data.data, data.target

# Преобразование в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

Шаг 3: Разделение на обучение и тестирование

```
# Разделите данные на обучающий и тестовый наборы  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

Шаг 4: Стандартизация данных

```
# Инициализация стандартного скейлера
scaler = StandardScaler()

# Подгонка и преобразование обучающих данных
X_train_scaled = scaler.fit_transform(X_train)

# Преобразование тестовых данных
X_test_scaled = scaler.transform(X_test)
```

Шаг 5: Обучение и оценка модели регрессии LASSO

```
# Инициализируем регрессионную модель LASSO с параметром регуляризации alpha
lasso = Lasso(alpha=0.1)

# Обучите модель на обучающих данных
lasso.fit(X_train_scaled, y_train)

# Делаем прогнозы на тестовых данных
y_pred_lasso = lasso.predict(X_test_scaled)

# Оценить модель
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print(f'Регрессия LASSO - средняя квадратичная ошибка: {mse_lasso}')
print(f'Регрессия LASSO - R-квадрат: {r2_lasso}')
```

Шаг 6: Обучение и оценка модели гребневой регрессии

```
# Инициализируем модель ридж-регрессии с параметром регуляризации alpha
ridge = Ridge(alpha=0.1)

# Обучить модель на обучающих данных
ridge.fit(X_train_scaled, y_train)

# Делаем прогнозы на тестовых данных
y_pred_ridge = ridge.predict(X_test_scaled)

# Оценить модель
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print(f'Ridge Regression - Mean Squared Error: {mse_ridge}')
print(f'Ridge Regression - R-квадрат: {r2_ridge}')
```


Шаг 7: Сравнение результатов

```
# Сравниваем коэффициенты регрессии LASSO и гребневой регрессии
coefficients = pd.DataFrame({
    'Feature': data.feature_names,
    'Коэффициенты LASSO': lasso.coef_,
    'Ridge Coefficients': ridge.coef_
})

print(coefficients)
```



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание



Проведите k-кратную перекрестную валидацию с помощью набора данных Iris, чтобы оценить эффективность модели машинного обучения. Для этой задачи мы будем использовать классификатор Support Vector Machine (SVM).

Пошаговая реализация:

1. Импорт необходимых библиотек
2. Загрузите набор данных Iris
3. Определите модель
4. Выполните 5-кратную перекрестную валидацию
5. Оценить модель



ВОПРОСЫ



Домашнее задание

1. Обучите модель логистической регрессии на наборе данных Breast Cancer. Оцените модель с помощью показателей точности, precision, recall и F1-score. Постройте кривую Precision-Recall.
2. Проведите кросс-валидацию с использованием [набора данных](#) (таргет - последний столбец) и модели логистической регрессии. Оцените модель с помощью оценок кросс-валидации и постройте кривую обучения.

Заключение

